PRODUCT INTRODUCTION · V8.11

GlassBox AIoPs Platform

以 AI 驅動的製程異常診斷平台
彈性 MCP/Skill 組合 × Claude LLM 深度分析

- 🔌 **彈性四層架構** — Event / Skill / MCP / DataSubject 自由組合
- 🤖 **六大 LLM 應用** — 腳本生成、參數對應、診斷邏輯、報告撰寫
- ⚡ **SSE 漸進式串流** — 每個 Skill 完成即時推播，不等待全部結果
- 🛡️ **安全沙箱執行** — RestrictedPython 防止惡意腳本危害系統

文件版本 v8.11　AI 引擎 Claude Opus 4.6　建立日期 2026-03-01

CHAPTER 1

系統定位與核心價值

> GlassBox AIoPs 解決半導體製造廠的核心痛點：製程異常發生後，PE 工程師需手動查詢多個系統、比對多筆資料才能診斷根因。GlassBox 將這套知識固化為可重用的診斷技能（Skill），由 AI 自動執行並生成報告。

傳統方式 vs. GlassBox AIoPs

✗ 傳統手動流程

- ✗ 接收警報後人工判斷要查哪些系統
- ✗ 登入多個系統逐一查詢 APC/EC/配方
- ✗ 依靠個人經驗比對資料判斷根因
- ✗ 撰寫診斷報告耗時且格式不一
- ✗ 資深 PE 知識無法系統化傳承

✓ GlassBox AIoPs

- ✓ 事件觸發後自動匹配診斷技能（Skill）
- ✓ MCP 自動查詢各系統資料來源
- ✓ Claude LLM 依診斷邏輯分析並得出結論
- ✓ 結構化報告含 Evidence + 建議行動
- ✓ Builder Copilot 固化 PE 知識為可配置 Skill

系統架構概覽

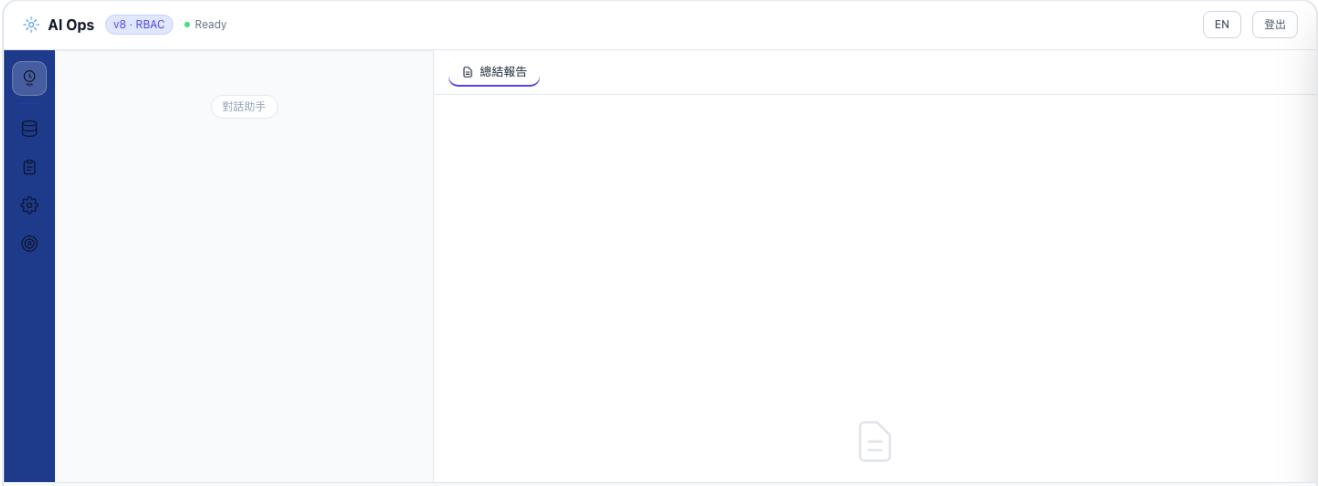

圖 1-1：GlassBox AIoPs 主介面 — 左側深藍導覽列 + 中央 30/70 診斷工作區

系統採用 **FastAPI** 後端（Python 3.11）+ 純 HTML/JS 前端（Tailwind CSS），以 **JWT Bearer Token** 做身份驗證，AI 分析由 **Anthropic Claude Opus 4.6** 驅動。

層面	技術選型	選擇理由
後端框架	FastAPI 0.115+	原生 async/await，支援 StreamingResponse SSE
資料庫	SQLite（dev）/ PostgreSQL（prod）	SQLAlchemy 2.0 AsyncSession，開發快速

層面	技術選型	選擇理由
AI 引擎	Claude Opus 4.6	最強推理能力，支援長 context + 結構化輸出
腳本沙箱	RestrictedPython	防止 AI 生成腳本危害系統安全
串流協定	Server-Sent Events (SSE)	單向服務端推播，比 WebSocket 更輕量
前端通訊	Fetch API + ReadableStream	支援設定 Authorization header（EventSource 不支援）

CHAPTER 2

彈性四層組件架構

> 系統核心設計為四層組件金字塔，每一層都可以獨立定義和組合，使 PE 工程師無需修改程式碼，即可靈活擴充新的診斷場景。

⚡ **Event Type**	定義「什麼事件觸發診斷」，包含事件名稱與屬性 Schema（如 equipment_id、measurement_value）。 • 一個 Event Type 可綁定多個 Skill • Event 發生時攜帶具體 params 值
🤖 **Skill**	定義「如何診斷」，綁定一個 MCP + `param_mappings`（事件屬性 → MCP 輸入）+ `diagnostic_prompt`（診斷邏輯）。 • 同一個 MCP 可被多個 Skill 共用 • param_mappings 由 AI Auto-Map 或手動定義
🔧 **MCP**	定義「如何取資料」，包含 input_params、output_schema、以及 AI 生成的 Python processing_script。 • MCP 腳本在 RestrictedPython 沙箱中執行 • AI 自動生成腳本，支援試執行驗證
🗄️ **DataSubject**	定義「資料在哪裡」，提供 API Endpoint（支援相對路徑與完整 URL）、HTTP Method 及 Response Schema。 • MCP 腳本執行時系統自動呼叫 DataSubject API 取得原始資料 • 可指向任意 REST API（工廠 MES、LIMS、SPC 系統等）

組合彈性示例

同一個「SPC OOC」事件，可同時觸發三個不同面向的診斷技能，每個 Skill 使用不同的 MCP 和資料來源：

Skill	MCP	DataSubject	診斷目的
Check APC Params	`mcp_check_apc_params`	APC_Data	APC 參數飽和度檢查
Check Recipe Offset	`mcp_check_recipe_offset`	Recipe_Data	配方最近修改稽核
Check Equipment Constants	`mcp_check_equipment_constants`	EC_Data	設備常數 vs 黃金基準

✅ 擴充新診斷場景只需三步

1. 定義新的 **DataSubject**（指向新的 API）
2. 在 **MCP Builder** 建立新 MCP，讓 AI 生成查詢腳本
3. 在 **Skill Builder** 建立新 Skill，綁定 MCP 並設定診斷邏輯

不需修改後端程式碼，PE 工程師透過 UI 即可完成全部設定。

CHAPTER 3

LLM 應用場景深度解析

> 系統在六個關鍵節點應用 Claude Opus 4.6，涵蓋從系統建置（Builder Copilot）到運行時診斷分析的完整生命週期，充分發揮 LLM 在語意理解和知識推理上的優勢。

六大 LLM 應用概覽

#	應用場景	觸發時機	API 端點	使用 Prompt
①	MCP 腳本生成	Builder：點擊「AI 生成腳本」	`/mcp-definitions/{id}/generate-script`	PROMPT_MCP_GENERATE
②	MCP 試執行驗證	Builder：點擊「試執行」	`/mcp-definitions/{id}/try-run`	PROMPT_MCP_TRY_RUN
③	Auto-Map 參數對應	Builder：點擊「AI Auto-Map」	`/builder/auto-map`	內建 Prompt
④	建議診斷邏輯	Builder：點擊「AI 建議邏輯」	`/builder/suggest-logic`	內建 Prompt
⑤	驗證診斷邏輯	Builder：點擊「Validate Logic」	`/builder/validate-logic`	內建 Prompt
⑥	診斷報告生成	執行診斷時（每個 Skill）	`/diagnose/event-driven-stream`	PROMPT_SKILL_DIAGNOSIS

① MCP 腳本自動生成

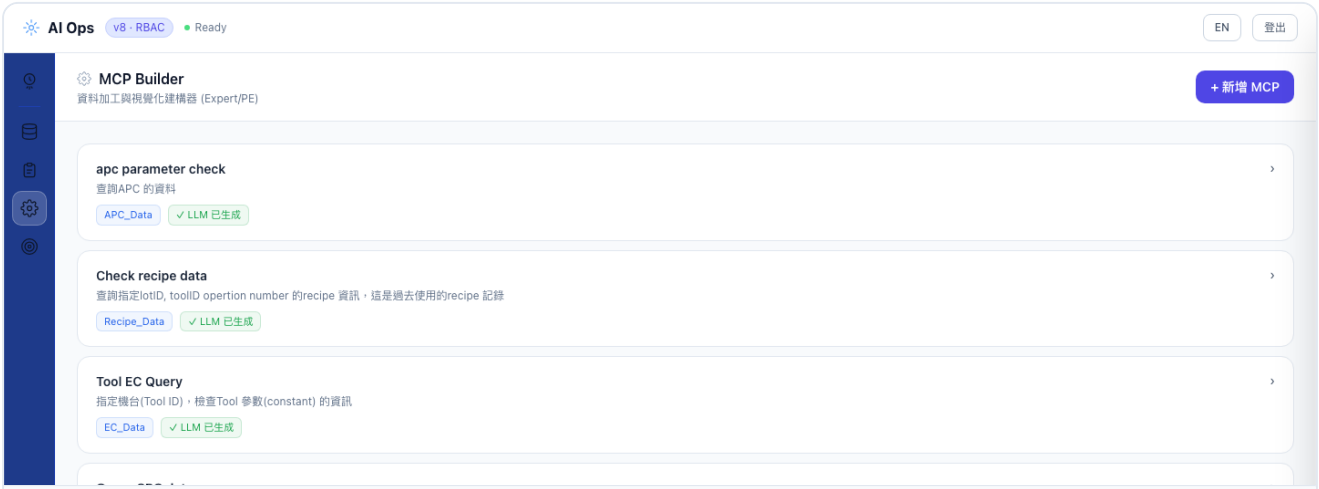

圖 3-1：MCP Builder — AI 根據 Input/Output Schema 自動生成安全的 Python 腳本

使用者只需在 MCP Builder 定義好 **input_params**（輸入參數 Schema）和 **output_schema**（輸出結構 Schema），點擊「🤖 AI 生成腳本」後，Claude 即依照以下規則自動生成 Python 腳本：

- 從 DataSubject API 回傳的 JSON 中提取所需欄位
- 進行必要的資料前處理（型別轉換、計算衍生值）
- 回傳符合 output_schema 定義的結構化字典
- 腳本嚴格遵循 RestrictedPython 安全規則（禁止 import os/sys/requests）

> 🔮 **AI 腳本生成的技術細節**
>
> 系統呼叫 `POST /api/v1/mcp-definitions/{id}/generate-script`，後端以 `PROMPT_MCP_GENERATE` 系統參數作為 System Prompt，將 MCP 的 input_params、output_schema 及 DataSubject 的 response_schema 組合為 User Prompt，交由 Claude Opus 4.6 生成腳本並寫回資料庫。

② MCP 試執行驗證

點擊「▶ 試執行」後，Claude（使用 `PROMPT_MCP_TRY_RUN`）自動：

1. 生成符合 input_params Schema 的測試資料
2. 呼叫 DataSubject API 獲取模擬回傳值
3. 在 RestrictedPython 沙箱中執行腳本
4. 比對輸出與 output_schema，回傳驗證報告

③ Auto-Map：語意感知的參數自動對應

Skill 需要定義 `param_mappings`，將事件的 params 欄位映射到 MCP 的 input_params。傳統方式需要 PE 工程師手動比對，容易出錯。**Auto-Map** 讓 AI 自動完成這個映射：

```
// Auto-Map 輸入
{
  "event_type_name": "SPC_OOC_Etch_CD",
  "skill_name": "check_apc_params",
  "mcp_name": "mcp_check_apc_params"
}

// Auto-Map 輸出（每個對應包含信心度與理由）
{
  "mappings":[
    {
      "mcp_param": "equipment_id",
      "source": "event_param",
      "event_field": "equipment_id",
      "confidence": "HIGH",
      "reasoning": "欄位名稱完全相同，語意明確對應"
    }
```

```
  ]
}
```

> ℹ️ **信心度評估（HIGH / MEDIUM / LOW）**
>
> Claude 根據欄位名稱相似性、說明語意、型別匹配度給出信心度評估，讓 PE 工程師可快速識別需要人工確認的低信心度對應項目。

④ Suggest Logic：PE 等級的診斷邏輯建議

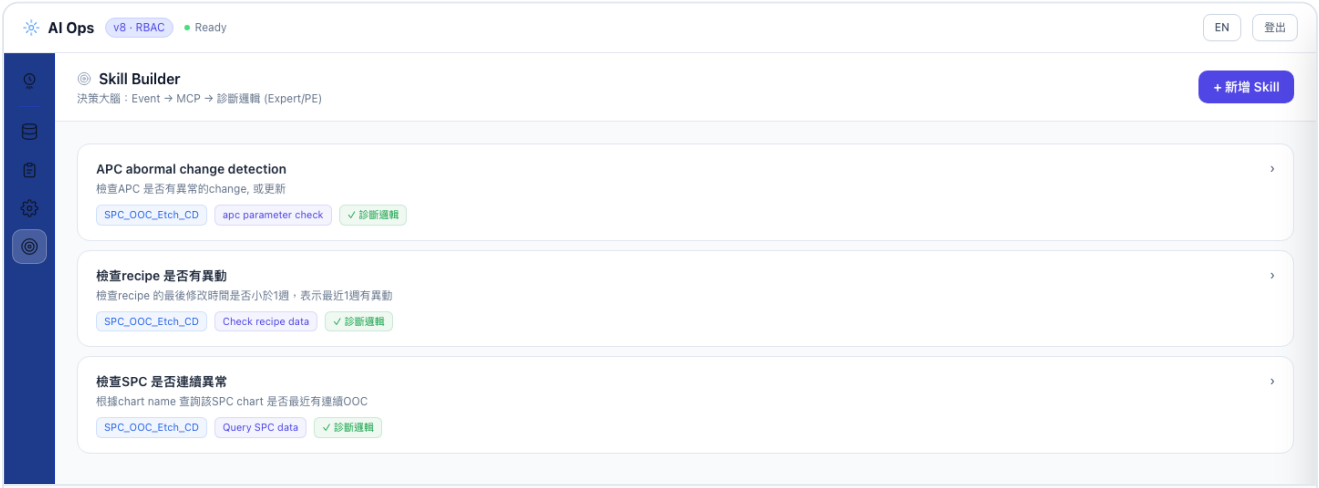

圖 3-2：Skill Builder — AI 建議邏輯功能可生成 PE 等級的診斷判斷條件

點擊「💡 AI 建議邏輯」後，Claude 以**資深半導體製程工程師**的角色，根據事件型別的屬性 Schema，自動生成 3–5 條業界標準的診斷邏輯建議，例如：

- 「當 `apc_saturation_level` 超過 85% 時，判定 APC 達到調整極限，應啟動 PM 評估」
- 「若 `recipe_modified_within_hours` ≤ 24 且 `modification_type` = 'OFFSET_CHANGE'，高度懷疑配方異動導致偏移」
- 「當 `ec_deviation_pct` > 15% 時，設備常數顯著偏離基準，建議安排 EC 校正確認」

⑤ Validate Logic：防止診斷邏輯引用不存在欄位

PE 工程師撰寫 `diagnostic_prompt` 時，可能引用不存在於 MCP output_schema 的欄位名稱，導致診斷執行時錯誤。**Validate Logic** 透過 Claude 自動檢查：

> ✅ **驗證通過**
>
> Prompt 中引用的所有欄位（如 `apc_saturation_level`）都存在於 MCP 的 output_schema 中，診斷執行時不會發生欄位缺失錯誤。

> ❌ **驗證失敗**
>
> Prompt 引用了不存在的欄位（如 `apc_usage_pct`），Claude 會指出錯誤欄位並建議正確的 Schema 欄位名稱。

⑥ 診斷報告生成：核心 AI 分析引擎

這是系統最核心的 LLM 應用，每次事件診斷時，對每個 Skill，系統執行以下流程：

1. **解析 param_mappings，提取事件屬性值**
   從事件的 params 字典中按 mapping 定義提取對應值，作為 MCP 的輸入

2. **呼叫 DataSubject API，獲取原始資料**
   系統自動 HTTP 請求 DataSubject 的 endpoint_url，傳入解析後的參數

3. **RestrictedPython 沙箱執行 MCP 腳本**
   對原始資料執行前處理，提取關鍵診斷指標，回傳符合 output_schema 的結構化資料

4. 🤖 Claude LLM
   依 diagnostic_prompt 生成診斷報告
   將 MCP 輸出資料 + diagnostic_prompt + PROMPT_SKILL_DIAGNOSIS 組合送入 Claude Opus 4.6，生成含 Evidence、Diagnosis、Recommendation 的結構化報告

5. **SSE 推播 skill_done 事件，前端即時渲染**
   診斷結果透過 Server-Sent Events 即時推播到前端，無需等待其他 Skill 完成

診斷報告結構

```
{
  "skill_name": "check_apc_params",
  "status": "ABNORMAL",      // NORMAL | ABNORMAL | ERROR
  "evidence": {              // MCP output_schema 資料
    "apc_saturation_level": 92.3,
    "apc_param_status": "SATURATED"
  },
  "diagnosis": "APC 飽和度 92.3% 超過警戒閾值...",
  "recommendation": "建議立即安排 PM 評估...",
  "mcp_output": { ... }        // 完整 MCP 原始輸出（含 dataset）
}
```

CHAPTER 4

SSE 漸進式串流診斷

> 系統採用 Server-Sent Events（SSE）實現「每個 Skill 完成即推播」的漸進式診斷體驗，使用者不必等待所有 Skill 完成才能看到結果。

事件觸發與診斷流程

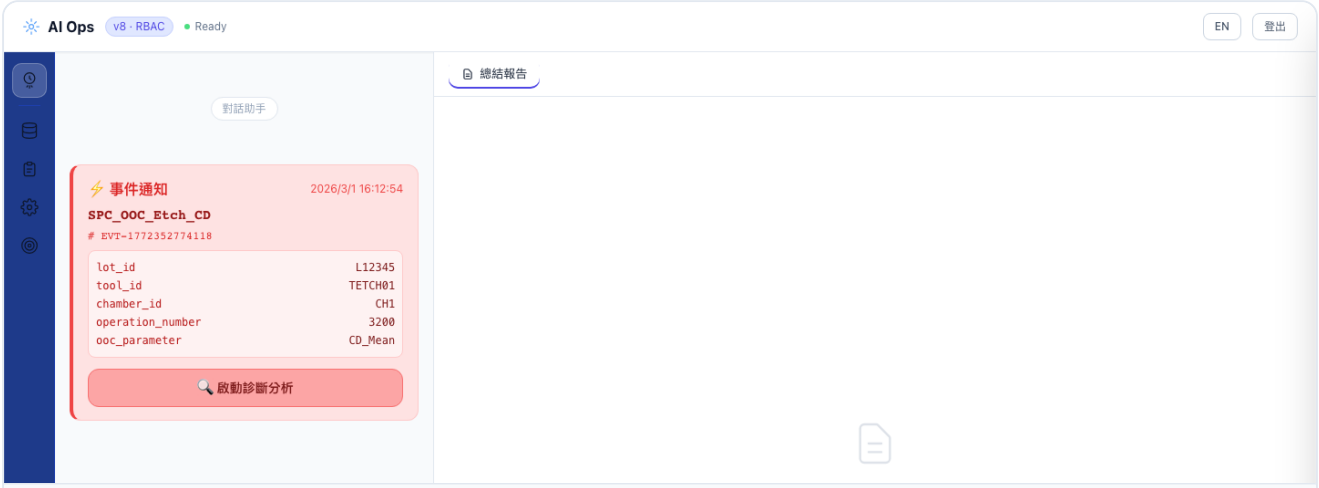

圖 4-1：紅色事件警告卡片，顯示 SPC OOC 事件的所有屬性與啟動診斷按鈕

SSE 事件格式

```
// 診斷開始
data: {"type": "start", "skill_count": 3, "event": {...}}

// Skill 開始執行（立即顯示 loading 狀態）
data: {"type": "skill_start", "skill_name": "check_apc_params", "index": 0}

// Skill 完成，立即渲染診斷卡片
data: {"type": "skill_done", "index": 0,
    "status": "ABNORMAL", "evidence": {...},
    "diagnosis": "APC 飽和度 92.3%...",
    "recommendation": "建議立即..."}

// 全部完成
data: {"type": "done", "total": 3}
```

前端 SSE 實作技術

前端使用 **Fetch API + ReadableStream**（而非 EventSource），原因是瀏覽器的 EventSource API 不支援設定自定義 Header（無法傳遞 JWT Token）：

```
const response = await fetch('/api/v1/diagnose/event-driven-stream',{
  method: 'POST',
  headers: {
```

```
    'Authorization': `Bearer ${token}`,
    'Content-Type': 'application/json'
  },
    body: JSON.stringify({ event_type, event_id, params })
});

const reader = response.body.getReader();
const decoder = new TextDecoder();
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  const lines = decoder.decode(value).split('\n');
  for (const line of lines) {
    if (line.startsWith('data: ')) {
      const event = JSON.parse(line.slice(6));
        handleSseEvent(event);  // 依 type 分派處理
    }
  }
}
```

30/70 分割布局設計

診斷工作區採用 **30% 聊天區 + 70% 報告區** 的分割設計：

左側 30%：聊天互動區

- 顯示系統訊息與進度提示
- Skill 開始執行時顯示「⏳ 正在執行...」
- Skill 完成時更新為「✅ 完成」
- 紅色事件卡片（事件屬性 + 啟動按鈕）

右側 70%：報告展示區

- 漸進式 Append 診斷卡片（Tab 形式）
- 每張卡片含：Evidence / Diagnosis / Recommendation
- 狀態圖示：NORMAL / ABNORMAL / ERROR
- 底部診斷摘要列（正常/異常/錯誤計數）

CHAPTER 5

Builder Copilot：AI 輔助知識建構

> Builder Copilot 是 GlassBox 的核心創新，讓 PE 工程師透過 AI 輔助，無需撰寫程式即可建構新的診斷場景。整個流程分為 MCP 建構和 Skill 建構兩個部分。

MCP 建構流程（A → B → C）

步驟	操作	AI 介入	結果
A. 定義	填入 MCP 名稱、說明、選擇 DataSubject；定義 input_params 和 output_schema	—	MCP 定義儲存至 DB
B. 生成腳本	點擊「🤖 AI 生成腳本」	Claude 根據 Schema 生成安全 Python 腳本 🤖 LLM	processing_script 填入 DB
C. 試執行	點擊「▶ 試執行」	Claude 生成測試資料並在沙箱執行 🤖 LLM	驗證 output_schema 正確性

Skill 建構流程（A → B → C → D → E）

步驟	操作	AI 介入	結果
A. 定義	填入 Skill 名稱、說明、選擇 MCP 和 Event Types	—	Skill 定義框架建立
B. Auto-Map	點擊「🤖 AI Auto-Map」	Claude 語意分析 Event 屬性 → MCP 輸入映射 🤖 LLM	param_mappings 自動填入（含信心度）
C. 確認 Mapping	人工審查 AUTO-MAP 結果，修正低信心度對應	—	param_mappings 最終確認
D. 設定診斷邏輯	點擊「💡 AI 建議邏輯」或手動撰寫	Claude 生成 PE 等級診斷邏輯建議 🤖 LLM	diagnostic_prompt 完成
E. 驗證邏輯	點擊「✅ Validate Logic」	Claude 確認 Prompt 欄位引用正確性 🤖 LLM	確保 Skill 可正常執行

🔮 **Builder Copilot 的設計哲學**

傳統 Expert System 需要程式師將 PE 的知識「硬編碼」進系統，知識更新成本極高。GlassBox 的 Builder Copilot 讓 **PE 工程師成為系統設計者**：他們定義「查什麼資料、用什麼邏輯診斷」，AI 負責「如何取資料、如何組織邏輯」，達到知識的可配置、可復用、可迭代。

CHAPTER 6

安全設計

> 系統在 AI 自動化的同時，通過多層安全機制確保企業資料安全和系統穩定性。

身份驗證與授權

機制	技術	說明
身份驗證	JWT Bearer Token	30 天有效期，HS256 簽名，儲存於 localStorage
密碼儲存	bcrypt（passlib）	含 salt 的雜湊儲存，不儲存明文
角色授權	三層 RBAC	it_admin / expert_pe / general_user，所有 API 端點均驗證
API 防護	FastAPI Depends	所有非公開端點均需 JWT，401 在 SSE 開始前返回

腳本沙箱安全

MCP 的 `processing_script` 由 AI 生成後在 **RestrictedPython** 沙箱中執行，防止惡意程式碼危害系統：

🚫 禁止操作

- `import os`（禁止系統操作）
- `import sys`（禁止進程控制）
- `import requests`（禁止外部網路）
- `import subprocess`（禁止命令執行）
- 檔案系統讀寫（open/write）

✅ 允許操作

- `import json`（資料解析）
- `import re`（正則表達式）
- `import datetime`（時間處理）
- `import math`（數學計算）
- 字典、列表、字串操作